Bloque 2: Simulación de Carga de Trabajo y Monitorización

Juan Reche Aguilar

21 de mayo de 2026

Índice

1. Ejercicio 4.4.1. Monitorización con Grafana + Prometheus.. 2
1.1. Guía con capturas. 2
1.2. Explicación general. 7

1. Ejercicio 4.4.1. Monitorización con Grafana + Prometheus..

1.1. Guía con capturas.

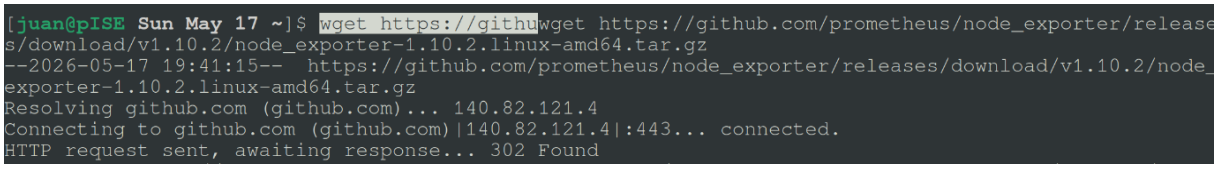


Figura 1: Descarga de Prometheus.

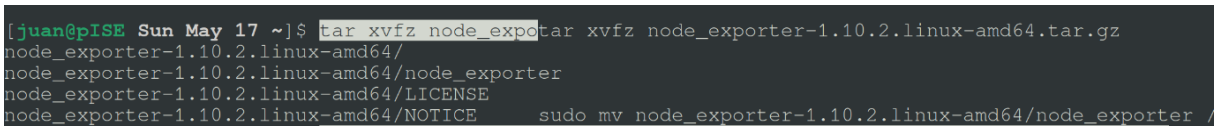


Figura 2: Descompresión del Exporter.



Figura 3: Movemos el programa a la carpeta de archivos binarios.

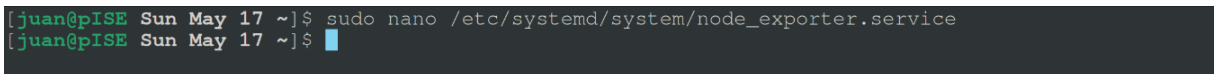


Figura 4: Archivo de configuración del sistema para node_exporter.

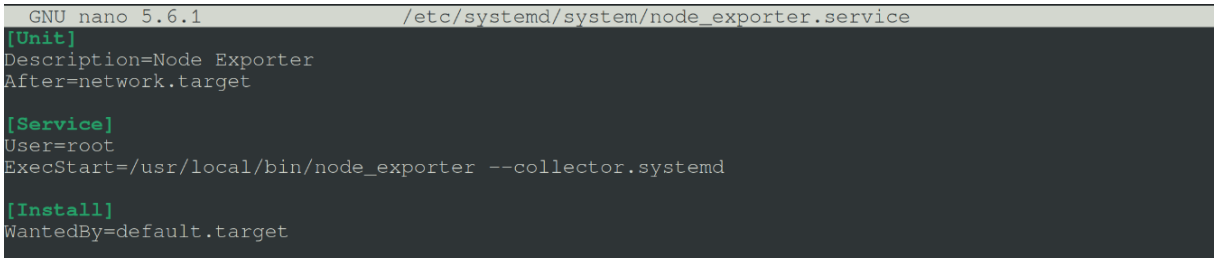


Figura 5: Configuración del node_exporter.

La intención de esta configuración es que el programa se ejecute automáticamente al iniciar el sistema, indicándole que se ejecute después de la interfaz de red, así como la carpeta donde se encuentra y sus permisos.

```
[juan@pISE Sun May 17 ~]$ sudo systemctl restart node_exporter                                  s
udo restorecon -v /usr/local/bin/node_exportsudo systemctl status node_exporter
● node_exporter.service - Node Exporter
     Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; vendor preset: disabled)
     Active: active (running) since Sun 2026-05-17 20:19:48 CEST; 6s ago
   Main PID: 11546 (node_exporter)
      Tasks: 3 (limit: 11109)
     Memory: 4.7M
        CPU: 8ms
     CGroup: /system.slice/node_exporter.service
             └─11546 /usr/local/bin/node_exporter --collector.systemd

May 17 20:19:48 pISE node_exporter[11546]: time=2026-05-17T20:19:48.910+02:00 level=INFO source=nod>
May 17 20:19:48 pISE node_exporter[11546]: time=2026-05-17T20:19:48.910+02:00 level=INFO source=nod>
May 17 20:19:48 pISE node_exporter[11546]: time=2026-05-17T20:19:48.910+02:00 level=INFO source=nod>
May 17 20:19:48 pISE node_exporter[11546]: time=2026-05-17T20:19:48.910+02:00 level=INFO source=nod>
May 17 20:19:48 pISE node_exporter[11546]: time=2026-05-17T20:19:48.910+02:00 level=INFO source=nod>
May 17 20:19:48 pISE node_exporter[11546]: time=2026-05-17T20:19:48.910+02:00 level=INFO source=nod>
May 17 20:19:48 pISE node_exporter[11546]: time=2026-05-17T20:19:48.910+02:00 level=INFO source=nod>
May 17 20:19:48 pISE node_exporter[11546]: time=2026-05-17T20:19:48.911+02:00 level=INFO source=tls>
May 17 20:19:48 pISE node_exporter[11546]: time=2026-05-17T20:19:48.911+02:00 level=INFO source=tls>
[juan@pISE Sun May 17 ~]$
```

Figura 6: Encendido y status de node_exporter.

Realmente es un reinicio debido a que ya se intento iniciarlo pero dio error debido a la red, para solucionarlo se ejecuto anteriormente:

```
sudo firewall-cmd --add-port=9100/tcp --permanent
sudo firewall-cmd --reload
```

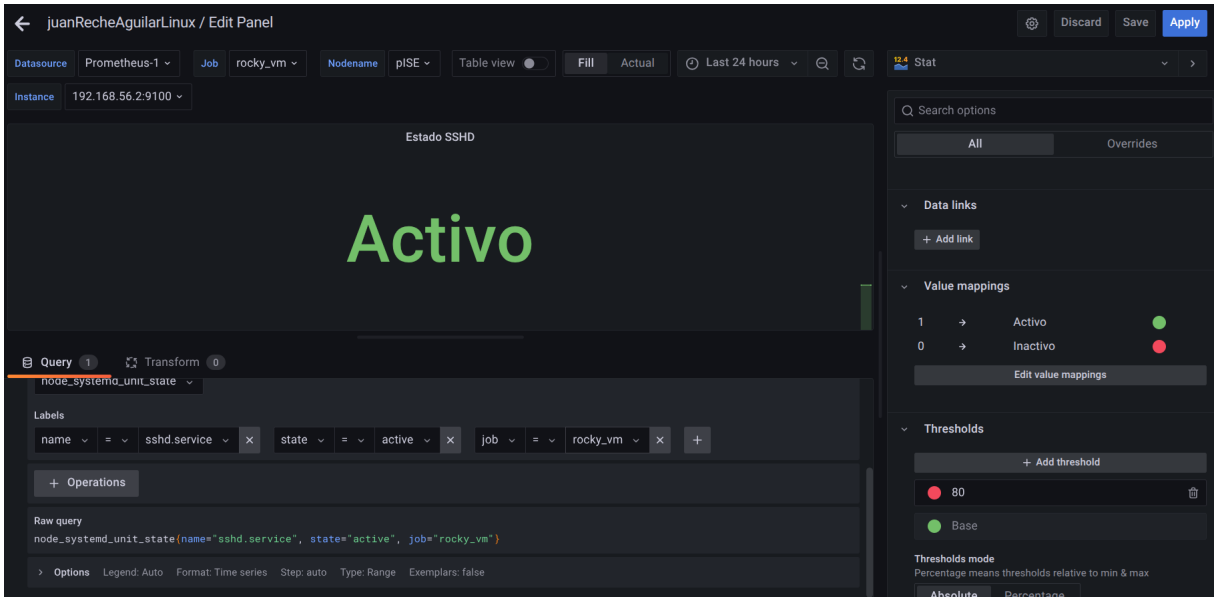


Figura 7: Creación del panel para monitorizar el puerto SSH (funcionamiento idéntico al de apache).



Figura 8: Ambos puertos activos.

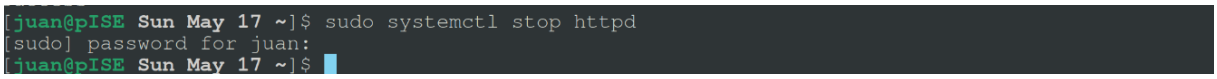


Figura 9: Detención del servicio HTTP.



Figura 10: Puerto SSH activo y puerto HTTP detenido.

Podemos ver como en un principio ambos puertos estaban activos, pero al detener el servicio HTTP, el puerto 80 se detuvo y el panel de Grafana lo reflejó.

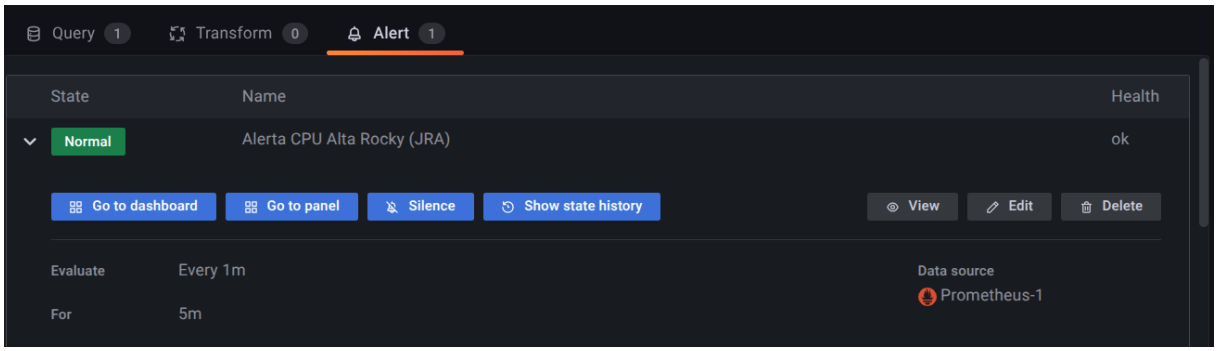


Figura 11: Alerta creada para el rendimiento del cpu (estado normal).

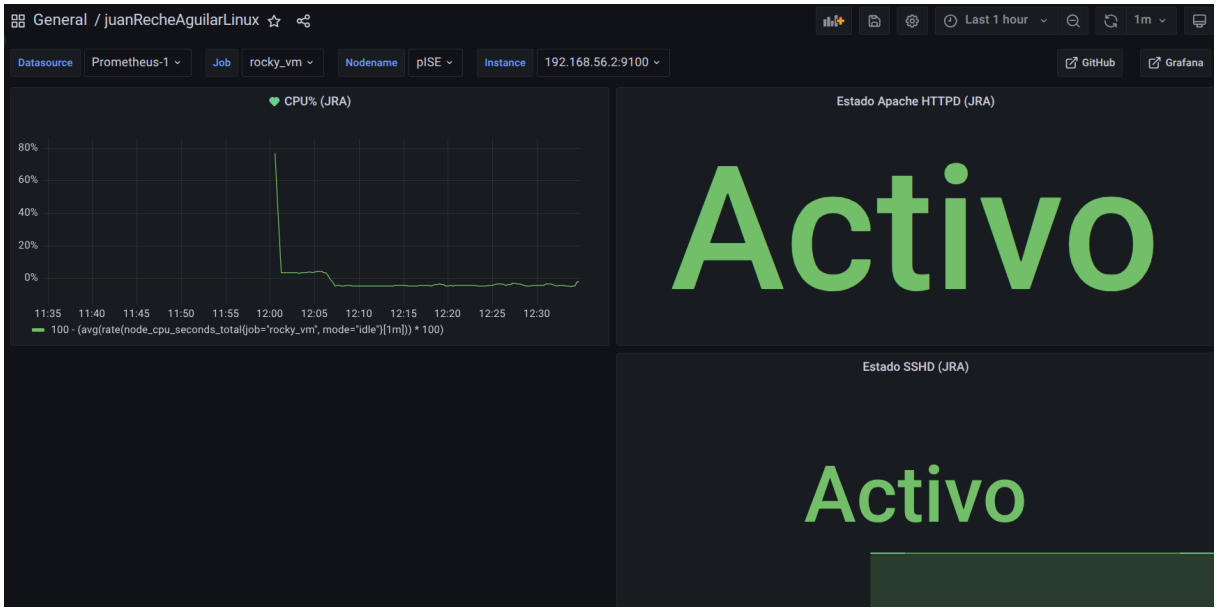


Figura 12: Estado de reposo del sistema antes de ejecutar stress. (esta captura se ha realizado después de realizar la practica para mostrar el consumo de CPU en reposo).

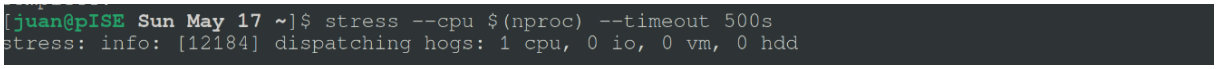


Figura 13: Ejecución del comando stress.

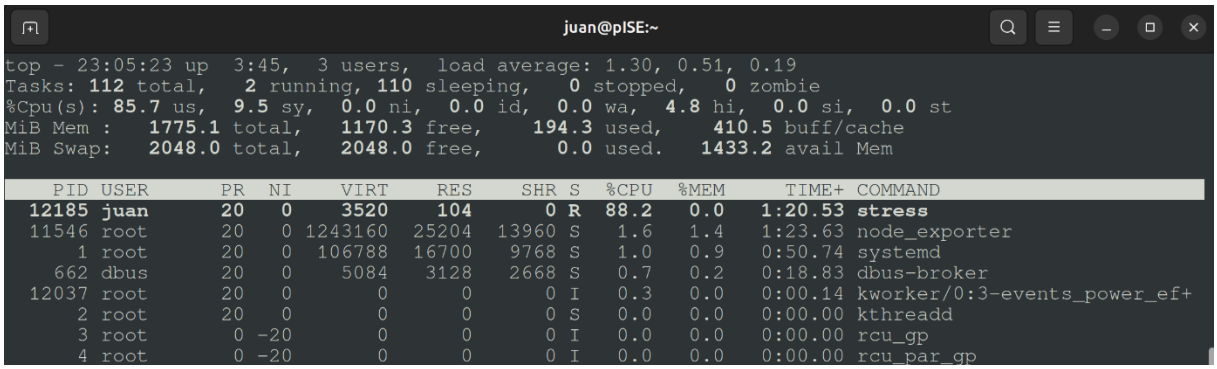


Figura 14: Verificamos que el comando stress funciona correctamente y está consumiendo recursos.

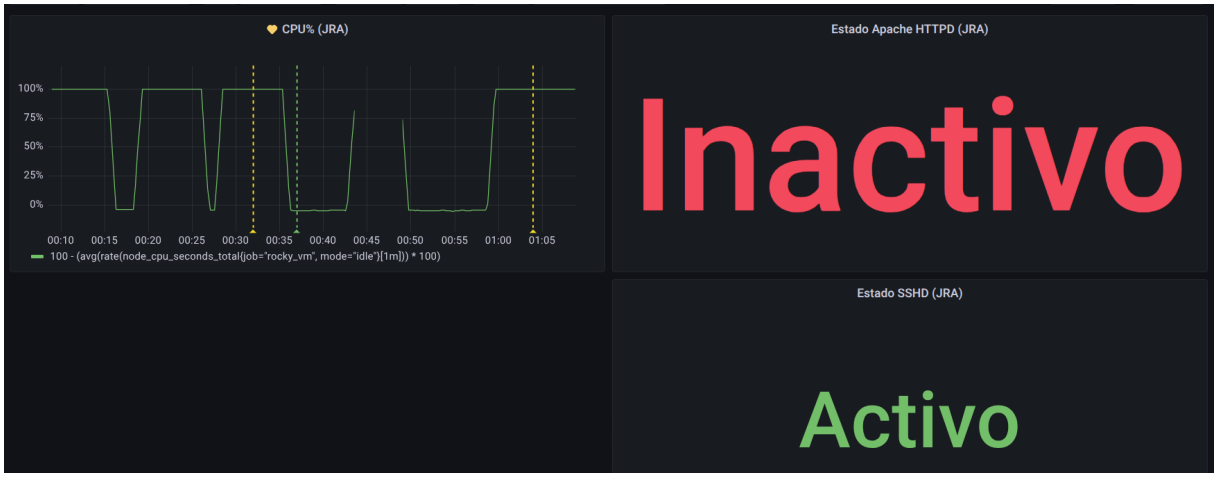


Figura 15: Acercándonos a la alerta.

Podemos ver como el consumo de CPU se ha disparado a causa del comando stress, y estamos a punto de alcanzar el umbral de la alerta que hemos creado.

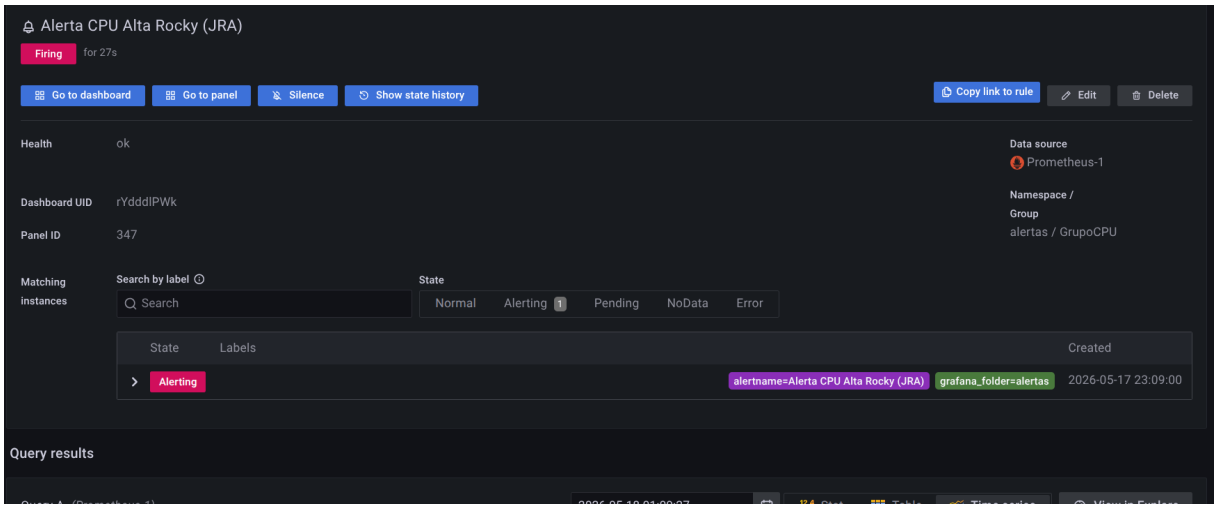


Figura 16: Alerta activada.

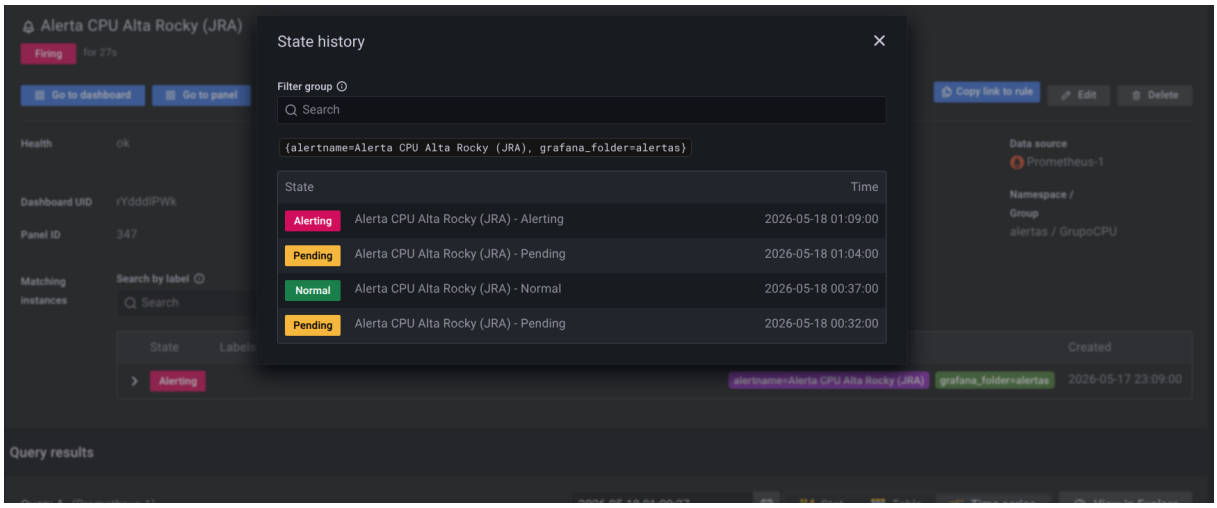


Figura 17: Un vistazo al historial de la alerta.

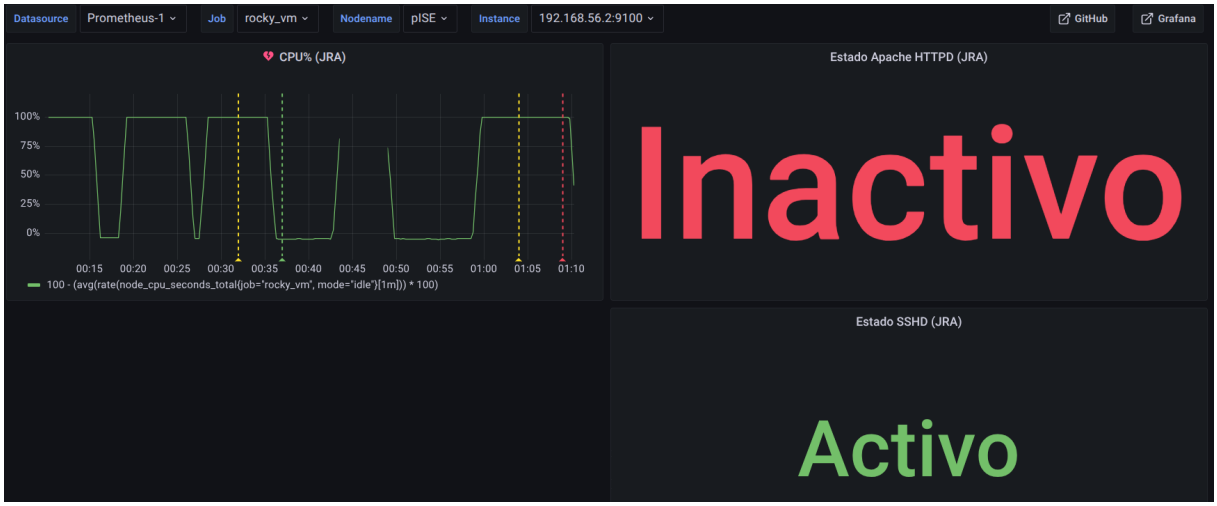


Figura 18: Después de saltar la alerta y terminar stress, el consumo de CPU ha bajado.

Se pueden observar varias subidas y bajadas del consumo de CPU debido a varias pruebas realizadas para activar la alerta.

1.2. Explicación general.

Para completar este ejercicio, se ha usado los archivos dados por el guión para levantar docker.
En una máquina virtual Rocky Linux se ha instalado Node Exporter, configurándolo como un servicio automático del sistema para que funcione de forma ininterrumpida en segundo plano. Aegurando antes que el programa tenga los permisos necesarios para ejecutarse y que el firewall permita el tráfico por el puerto 9100, que es el que usa el programa para enviar la información a Prometheus.
Después de las configuraciones de prometheus, se paso a configurar los paneles del dashboard de Grafana. Se han creado dos paneles para monitorizar el puerto 80 (HTTP) y el puerto 22 (SSH), comprobando que funcionaran al apagar apache y que mostrara los datos correspondientes.

Por ultimo se creó una alerta para el consumo de CPU, con un umbral del 75 %, y se ha ejecutado el comando stress para simular una carga de trabajo que dispare la alerta. Se realizó varias veces de ahí que en la captura de la gráfica y en el historial se puedan observar varias subidas y bajadas del consumo de CPU, fueron errores de poco tiempo del programa stress y de malas configuraciones del tiempo que la alerta tomaba como media de tiempo para el calculo del uso de CPU. Una vez solucionados estos errores, se pudo activar la alerta correctamente y observar su funcionamiento.